

Fine-Tuning DistilBERT for Sentiment Classification

Technical Overview & System Notes

Overview

This project sets up a sentiment analysis pipeline by fine-tuning `distilbert-base-uncased` on movie review data from the IMDb dataset. Using PyTorch alongside Hugging Face's `transformers`, `datasets`, and `evaluate` packages, the pre-trained encoder model is updated to classify input text as either positive or negative.

Project Setup & Stack

Component	Choice	Notes
Model	distilbert-base-uncased	Pre-trained Transformer (66M parameters)
Framework	PyTorch / Hugging Face Transformers	Model training and pipeline inference
Dataset	IMDb Reviews	Binary classification dataset (0: negative, 1: positive)
Metrics	Accuracy (Hugging Face Evaluate)	Evaluates argmax predictions against ground truth

Implementation Code

```
import numpy as np
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
    pipeline
)
import evaluate

# Load dataset and prepare tokenizer
dataset = load_dataset("imdb")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def tokenize_function(example):
    return tokenizer(example["text"], padding="max_length", truncation=True,
```

```

max_length=256)

tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Select smaller samples for faster execution
small_train = tokenized_dataset["train"].shuffle(seed=42).select(range(1000))
small_test = tokenized_dataset["test"].shuffle(seed=42).select(range(500))

# Load pre-trained model and metric
model = AutoModelForSequenceClassification.from_pretrained("distilbert-base-uncased",
num_labels=2)
accuracy = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=1)
    return accuracy.compute(predictions=predictions, references=labels)

# Configure training hyperparameters
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_steps=50
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=small_train,
    eval_dataset=small_test,
    compute_metrics=compute_metrics
)

# Train and evaluate
trainer.train()
results = trainer.evaluate()
print(results)

# Test inference on sample sentences
classifier = pipeline("sentiment-analysis", model=trainer.model, tokenizer=tokenizer)
sentences = [
    "The movie was absolutely wonderful!",
    "The film was boring and disappointing.",
    "I love this product.",
    "The service was terrible.",
    "The food was delicious.",
    "The laptop performance is excellent.",
    "I am not satisfied with the quality.",
    "This is the best experience I have ever had."
]

predictions = classifier(sentences)

```

```
for sentence, prediction in zip(sentences, predictions):  
    print("Sentence :", sentence)  
    print("Prediction:", prediction)  
    print("-" * 60)
```

Process Notes

- **Preprocessing:** Text sequence lengths are capped at 256 tokens to manage memory during tokenization. For initial validation, the dataset was trimmed to 1,000 training examples and 500 test examples.
- **Training Setup:** The learning rate is set to `2e-5` with weight decay of `0.01` across 2 epochs using a batch size of 8.
- **Inference:** The fine-tuned model is passed directly into Hugging Face's `pipeline` for single and batch predictions.

Next Steps

- Train on the full 25,000 example training set rather than the subset to improve real-world generalization.
- Extend training to 3 or 4 epochs while monitoring evaluation loss to check for potential overfitting.